

Guía de Integración: Frontend (React) + Backend (FastAPI/TiDB)

Proyecto: Sistema Evaluador Inteligente Tipo CENEVAL **Arquitectura:** Motor de Inferencia Blindado (Lógica en el Backend)

1. Resumen de la Arquitectura

Para este proyecto, hemos implementado una **Estrategia Blindada**. El Frontend (React) no gestiona la lógica del examen (niveles, aciertos o qué pregunta sigue); su única responsabilidad es **presentar la información** y **enviar las interacciones del usuario**. El Backend (Python) actúa como el Agente Inteligente que mantiene el estado y decide el camino del alumno.

2. Base URL del API

- **Local:** `http://127.0.0.1:8000`
- **Producción:** (Se definirá al desplegar en Render/Vercel)

Nota: El Backend ya tiene habilitado CORS para permitir peticiones desde cualquier origen en desarrollo.

3. Flujo de Trabajo del Alumno (Sprints de Frontend)

Paso 1: Inicialización

Al cargar la aplicación o presionar "Iniciar Examen", el Frontend debe solicitar un ID de sesión único.

- **Endpoint:** `GET /iniciar`
- **Respuesta:** ```json { "id_sesion": "a1b2c3d4", "mensaje": "Examen iniciado" }`
- **Acción:** Guardar `id_sesion` en el estado global (Context/Redux) o en un `useState` principal.

Paso 2: Obtener la Pregunta Actual

El Frontend nunca pide "la pregunta 5" o "una pregunta difícil". Solo pide "la pregunta que me toca".

- **Endpoint:** `GET /pregunta/{id_sesion}`
- **Respuesta (Éxito):**
- JSON

```
{
  "id_pregunta": 1,
  "contexto": "Escenario del caso de estudio...",
  "pregunta_texto": "¿Qué dispositivo es el adecuado?",
  "opciones": [
    {"id_opcion": 10, "texto_opcion": "Switch"},
    {"id_opcion": 11, "texto_opcion": "Router"}
  ],
  "progreso": 5
}
```

-
-
- **Respuesta (Examen Terminado):**
- JSON

```
{
  "finalizado": true,
  "mensaje": "Examen terminado. Revisa tus resultados."
}
```

-
-

Paso 3: Validar Respuesta

Cuando el usuario selecciona una opción y da clic en "Enviar/Siguiente".

- **Endpoint:** `POST /validar`
- **Cuerpo (JSON):**
- JSON

```
{
  "id_sesion": "a1b2c3d4",
  "id_pregunta": 1,
  "id_opcion": 10
}
```

-
-
- **Respuesta del Agente Inteligente:**
- JSON

```
{
  "es_correcta": true,
  "retroalimentacion": "¡Excelente! Los switches son plug-and-play...",
  "instruccion_agente": "¡Dominas el tema! Avanzando al siguiente nivel.",
  "progreso": "6/30"
}
```

-
-
- **Acción:** Mostrar la retroalimentación al alumno y luego llamar nuevamente al Paso 2 para obtener la siguiente pregunta.

4. Reglas del Motor de Inferencia (Lógica del Backend)

El equipo de Frontend debe saber que el Backend aplica estas reglas automáticamente:

1. **Avance de Tema:** Si el alumno tiene **3 aciertos consecutivos**, el Backend cambia automáticamente el tema y reinicia la dificultad a 1.
2. **Adaptabilidad:** Si el alumno falla, el Backend incrementa la dificultad (máximo nivel 3) en la siguiente pregunta del mismo tema.
3. **Límite de Seguridad:** Al llegar a **30 preguntas**, el endpoint de pregunta devolverá `finalizado: true`.

5. Recomendaciones para el Frontend (Claude Tips)

- **Consumo de API:** Usar `fetch` o `axios`.
- **UI Dinámica:** Se recomienda usar una tarjeta de visualización para el `contexto` (caso de estudio) separada de la `pregunta_texto`.
- **Estados de Carga:** Implementar un Spinner mientras se espera la respuesta de `/validar` o `/pregunta`, ya que el motor de inferencia consulta la base de datos en la nube (TiDB).
- **Manejo de Errores:** Si el API responde un error 500, mostrar un mensaje de "Error de conexión con el Agente Inteligente".